AtlasDB: Design and Implementation of a From-Scratch Embedded Relational Database Engine in Go

Harshit Nakrani

Independent Systems Engineering Project

Abstract—AtlasDB is a research and engineering project aimed at implementing a small relational database engine from first principles in Go. The project is motivated by the observation that database abstractions hide a large amount of systems engineering: persistent pages, buffer management, ordered indexes, record formats, query execution, transactions, logging, recovery, and concurrency. Rather than treating SQL as the starting point, AtlasDB is designed bottom-up, beginning with raw file I/O and a page-oriented storage layer and progressing toward a B+ tree storage engine, relational execution, and transactional durability. This paper presents the motivation, design goals, architecture, storage model, B+ tree design, record representation, query-processing pipeline, transaction and recovery strategy, concurrency model, implementation plan, and evaluation methodology for AtlasDB. The project deliberately separates established database concepts from future experimental features so that performance claims are not made before the corresponding implementation and benchmarks exist. AtlasDB is therefore positioned as both a functional database prototype and an experimental platform for studying database internals.

Keywords— database systems, storage engine, B+ tree, pager, buffer pool, WAL, Go, relational database, query execution

Project status: architecture and implementation in progress; experimental results are intentionally not fabricated.

1. Introduction

A database system is often encountered through a simple interface: a client submits a SQL statement and receives rows in return. The apparent simplicity of this interface hides a layered storage and execution system. Persistent data must be represented as bytes, bytes must be organized into pages, pages must be cached and modified safely, indexes must support ordered lookup, records must be serialized, queries must be transformed into executable plans, and failures must not leave the database in an invalid state. The engineering difficulty is therefore not the SQL syntax itself, but the set of invariants connecting the logical and physical layers.

AtlasDB is designed as a from-scratch implementation intended to make those layers explicit. The project uses Go because it provides direct access to files, byte buffers, synchronization primitives, and profiling facilities while keeping the implementation compact enough for a single developer to understand. The choice of Go is also consistent with the project's broader interest in systems and backend engineering.

The design is informed by classic B-tree research, modern database literature, and the architecture of established systems such as SQLite. SQLite is particularly useful as a reference because its database file is page-oriented, its B-tree structures are persistent, and its pager forms a distinct abstraction between higher-level database logic and the operating system. SQLite documentation explicitly describes the database as a collection of uniform pages and identifies B-tree pages as the fundamental structure used for table and index storage [1], [2].

AtlasDB does not attempt to reproduce SQLite's exact file format or compatibility guarantees. Instead, it uses SQLite and the literature as architectural references while making implementation decisions that are easier to inspect, change, benchmark, and explain.

2. Research Objectives

The project has six primary objectives:

- Implement a persistent page-oriented storage engine from raw file I/O.
- Implement an ordered B+ tree suitable for persistent table and index storage.
- Develop a compact record format and a schema/catalog layer.
- Build a minimal relational query pipeline from SQL parsing to execution.
- Add write-ahead logging, transaction semantics, and crash recovery.
- Create an observable and benchmarkable engine that exposes internal behavior rather than treating the storage engine as a black box.

A secondary objective is educational: each subsystem should have an explicit rationale, tests, and measurements. The project therefore treats architecture diagrams, design records, benchmarks, and failure tests as part of the implementation rather than as documentation added after the fact.

3. Background and Related Work

3.1 B-tree and B+ tree indexing

Bayer and McCreight introduced the B-tree as a structure for maintaining large ordered indexes on storage devices where access to a page is substantially more expensive than computation in main memory [3]. Their analysis emphasizes retrieval, insertion, and deletion with logarithmic dependence on index size while maintaining high page utilization. The central idea is to organize keys into multi-way nodes whose contents are kept ordered and whose children point toward progressively narrower key ranges.

A B+ tree follows the same page-oriented principle but separates routing information from the records stored at the leaves. Internal nodes contain separator keys and child pointers, while leaf nodes contain the searchable entries. Leaf-level sibling links make ordered and range traversal efficient. In a disk-backed implementation, the high fan-out of each node keeps tree height small, which reduces the number of page accesses needed for a point lookup.

3.2 SQLite as a reference architecture

SQLite provides a useful reference implementation for AtlasDB because its file format defines pages, B-tree page types, cell layouts, overflow pages, and a schema table. SQLite's documentation states that all pages in a database have the same page size and that reads and writes to the main database normally operate on page boundaries [1]. SQLite's B-tree pages contain interior and leaf variants, with interior pages storing keys and child pointers and leaf pages storing table data or index entries [1].

The SQLite pager is intentionally separated from B-tree and SQL details. Its documentation describes the pager as responsible for page-level access, caching, locking, and the atomicity and durability mechanisms required by the database [4]. AtlasDB adopts the same broad separation while keeping the first implementation substantially simpler.

3.3 Query processing and analytical systems

SQLite's query planner demonstrates the importance of separating declarative SQL from physical execution. A SQL statement specifies the desired result while the planner chooses among algorithms such as full scans and indexed lookups [5]. AtlasDB will follow the same conceptual separation: parsing produces an abstract representation, planning chooses an execution strategy, and the executor performs that strategy.

Column-oriented database research provides a useful contrast. Column stores reduce unnecessary data movement for analytical queries by storing attributes separately and reading only the columns required by a query [6]. AtlasDB's initial engine will use a row-oriented representation because it is simpler for a first persistent relational engine, but the architecture will leave room for future experimental columnar or hybrid storage.

4. Design Principles

- Bottom-up construction: storage primitives are implemented before SQL features.
- Explicit invariants: each subsystem defines what must remain true before and after an operation.

- Minimal dependencies: the initial engine relies primarily on the Go standard library.
- Deterministic behavior where practical: reproducible tests are preferred over hidden heuristics.
- Measure before optimizing: performance work is driven by benchmarks and profiling.
- Observability: internal page, tree, WAL, and execution state should be inspectable.
- Failure-aware design: crash recovery and partial operations are considered during subsystem design rather than added at the end.

5. System Architecture

AtlasDB is organized around database subsystems rather than conventional web-application layers. The root project contains command-line entry points, documentation, tests, benchmarks, examples, and scripts. The private database engine resides under internal/. This keeps storage and execution details inaccessible to unrelated packages and encourages a clear boundary between the engine and its command-line interface.

Figure 1. Proposed AtlasDB repository organization.

```
atlasDB/
├── benchmarks/
├── cmd/
│   └── atlas/
│       └── main.go
├── data/
├── docs/
├── examples/
├── internal/
│   ├── common/
│   ├── storage/
│   ├── pager/
│   ├── buffer/
│   ├── btree/
│   ├── catalog/
│   ├── record/
│   ├── parser/
│   ├── planner/
│   ├── executor/
│   ├── wal/
│   ├── recovery/
│   ├── transaction/
│   ├── concurrency/
│   ├── cache/
│   ├── config/
│   └── engine/
├── pkg/
├── scripts/
├── tests/
└── go.mod
```

The logical dependency direction is intentionally downward for storage operations and upward for query operations. A typical query follows a path from the engine through the parser, planner, executor, and storage structures, while the physical path from a persistent record proceeds through a B+ tree, buffer manager, pager, and raw file storage.

Figure 2. Simplified AtlasDB execution and storage path.

```
SQL
 |
Parser
 |
Planner
 |
Executor
 |
B+ Tree / Table / Index
 |
Buffer Manager
 |
Pager
 |
Storage
 |
atlas.db
```

6. Persistent Storage Layer

The first AtlasDB subsystem is raw storage. Its responsibility is deliberately narrow: open a database file and perform byte-oriented reads and writes at explicit offsets. It does not know what a page, record, B+ tree, or SQL statement means. This separation prevents the lowest layer from becoming coupled to higher-level database concepts.

6.1 Storage API

Code Example 1. Initial raw storage interface in Go.

```
type File struct {
    file *os.File
}


func Open(path string) (*File, error)
func (f *File) ReadAt(offset int64, buf []byte) error
func (f *File) WriteAt(offset int64, data []byte) error
func (f *File) Sync() error
func (f *File) Size() (int64, error)
func (f *File) Close() error
```

The storage layer uses positional I/O so higher layers can address independent regions of the file without maintaining a shared file cursor. The implementation can be backed by os.File and the ReadAt and WriteAt operations provided by the Go standard library. Sync is exposed explicitly because durability requires a defined relationship between a successful database operation and persistence to stable storage.

7. Pages and the Pager

The pager is the first layer that understands database pages. A page is a fixed-size block that provides the unit of physical I/O and the unit around which the buffer manager and B+ tree operate. SQLite similarly models its database as uniformly sized pages and performs page-oriented access [1], [4]. AtlasDB will initially use a configurable page size, with 4096 bytes as a practical starting point for experiments.

If page identifiers begin at zero, the byte offset of page p is p multiplied by the configured page size. The pager therefore converts a logical PageID into a storage offset and enforces page-sized reads and writes.

Code Example 2. Simplified page addressing and reading.

```
const PageSize = 4096

type PageID uint64

type Page struct {
    ID   PageID
    Data []byte
}

func offset(id PageID) int64 {
    return int64(id) * PageSize
}

func (p *Pager) ReadPage(id PageID) (*Page, error) {
    buf := make([]byte, PageSize)
    if err := p.file.ReadAt(offset(id), buf); err != nil {
        return nil, err
    }
    return &Page{ID: id, Data: buf}, nil
}
```

The pager is intentionally separate from the storage package. A future change from 4 KiB to 8 KiB pages should alter page semantics without requiring raw file I/O to understand pages. This separation also provides a natural location for page allocation, dirty-page tracking, and later buffer-pool integration.

8. Buffer Manager

Directly reading every requested page from storage would make the database unnecessarily dependent on storage latency. The buffer manager keeps a bounded set of pages in memory and decides which pages can be reused when the pool is full. The initial implementation can use a simple replacement policy, followed by LRU or clock-based policies once correctness is established.

A buffer frame must track at least the page identifier, whether the page is dirty, and whether a client currently pins the page. A dirty page cannot be safely discarded until its contents have been written according to the database's durability protocol. Pinning prevents an in-use page from being selected for eviction.

Code Example 3. Simplified buffer-pool state.

```
type Frame struct {
    PageID PageID
    Data   []byte
    Dirty  bool
    Pin    uint32
}

type BufferPool struct {
    frames map[PageID]*Frame
    // replacement metadata is added later
}
```

9. Record and Page Formats

A database cannot persist Go structs directly and expect the resulting bytes to remain a stable database format. AtlasDB therefore requires an explicit serialization format. A record format defines how values such as integers, strings, booleans, and nullable fields become bytes. The format should be versionable and independent of Go's memory layout.

A first record format can use a fixed header followed by a sequence of encoded values. For example, the header may contain the number of columns and offsets or lengths for variable-sized fields. A more compact format can use variable-length integers for values whose magnitude is usually small. SQLite uses a record format and variable-length integers to represent table and index payloads [1]. AtlasDB does not need to copy that format, but the same general principle—explicit byte-level representation—is applicable.

Code Example 4. Record serialization boundary.

```
// Illustrative, not the final on-disk format.
type Record struct {
    Values []Value
}

func (r *Record) Encode(dst []byte) (int, error) {
    // 1. encode column count
    // 2. encode per-value type information
    // 3. encode value payloads
    return 0, nil
}
```

10. B+ Tree Storage Engine

The B+ tree is the central ordered storage structure in the first AtlasDB design. Its purpose is not merely to make search fast; it also organizes persistent data into a structure whose nodes can be mapped naturally onto pages. A large fan-out means that a tree can represent a large number of keys with relatively small height.

An internal node stores separator keys and child references. The separator keys guide traversal to the appropriate child. Leaf nodes store the actual entries and are linked to neighboring leaves so a range scan can continue from one leaf to the next without repeatedly returning to the root. The exact separator convention will be defined by the AtlasDB implementation and tested as an invariant.

10.1 Search

Code Example 5. Conceptual B+ tree point lookup.

```
func (t *Tree) Search(key Key) (Value, bool) {
    page := t.root

    for !page.IsLeaf() {
        i := page.FindChild(key)
        page = t.loadChild(page, i)
    }

    i := page.FindKey(key)
    if i < 0 {
        return Value{}, false
```

```
    }
    return page.ValueAt(i), true
}
```

The important property is that a lookup visits a path from the root to a leaf rather than scanning all records. The number of page visits grows with tree height. Because each internal page can contain many separators and child references, tree height grows slowly as the number of records increases.

10.2 Insertion and splitting

Insertion begins by locating the target leaf. If the leaf has sufficient free space, the entry can be inserted in sorted order. If the page is full, AtlasDB will split the page into two nodes, redistribute entries, and propagate a separator into the parent. If the parent is also full, the split can propagate upward. A root split creates a new root and increases tree height.

This recursive propagation is the mechanism that preserves B+ tree invariants after growth. The implementation will test sorted keys, minimum and maximum occupancy, correct parent-child relationships, and equal leaf depth.

10.3 Deletion and rebalancing

Deletion is more complex because removing an entry can reduce a node below its minimum occupancy. The implementation will support redistribution with a sibling when possible and merging when necessary. A merge can require deletion of a separator from the parent and can therefore propagate upward. Root contraction is handled as a special case.

11. Catalog and Schema

The catalog maps logical database objects to their physical representation. At minimum, it must represent tables, columns, and indexes and identify the root page of each persistent structure. This is conceptually similar to SQLite's schema table, which records database objects and their root pages [1].

The catalog must itself be persistent. A database that loses its table definitions after restart is not usable even if the underlying data pages remain intact. AtlasDB will therefore store catalog records in a reserved system structure or system table once the basic B+ tree and record layers are stable.

12. SQL Parsing, Planning, and Execution

SQL is introduced only after the storage engine can reliably persist and retrieve records. This sequencing prevents the parser from becoming a distraction from the underlying database mechanics.

12.1 Parser

The parser converts a token stream into an abstract syntax tree (AST). The initial grammar will be intentionally small, for example CREATE TABLE, INSERT, SELECT with simple predicates, UPDATE, DELETE, and basic ORDER BY or LIMIT support. The AST should represent semantics rather than retain every syntactic detail.

Code Example 6. Illustrative SQL AST node.

```
type SelectStmt struct {
    Columns []Expr
    Table   string
    Where   Expr
    Limit   int
}
```

12.2 Planner

The planner converts an AST into an execution plan. A simple implementation can choose between a full table scan and an index lookup. Later versions can add cost estimates, multi-column indexes, join ordering, predicate pushdown, and covering-index decisions. SQLite's query planner documentation illustrates why declarative SQL requires a planning layer: the same logical query can be executed through multiple physical algorithms [5].

12.3 Executor

The executor implements physical operators such as table scan, index scan, filter, projection, aggregation, and join. Operators should expose a small iterator-like interface initially. A vectorized execution model may be investigated later if analytical workloads become a project goal.

13. Transactions, WAL, and Recovery

A database must remain consistent across process termination and hardware or operating-system failures. AtlasDB will use write-ahead logging (WAL) as the primary durability mechanism. Before a dirty page is written to the main database, the required log information must satisfy the WAL ordering rule: the log describing a change reaches durable storage before the corresponding data page is allowed to become durable.

SQLite uses WAL as one of its journaling modes, and its broader pager architecture demonstrates the importance of keeping durability and page management explicit [4], [7]. AtlasDB's first WAL implementation will be intentionally small: append records describing page-level changes, mark transaction boundaries, and replay committed changes during recovery.

Code Example 7. Simplified WAL record model.

```
type LogRecord struct {
    TxID   uint64
    PageID PageID
    Offset uint16
    Before []byte
    After  []byte
}

type WAL struct {
    file *os.File
}

func (w *WAL) Append(r LogRecord) error {
    // encode record -> append -> sync
    return nil
}
```

13.1 Recovery

Recovery will scan the WAL after an unclean shutdown, determine which transactions committed, and restore pages to a consistent state. The exact recovery algorithm will be selected after the page and WAL formats are stable. Recovery tests will deliberately terminate the process at selected points in an operation and verify that reopening the database does not violate structural invariants.

14. Concurrency Control

The initial AtlasDB implementation will prioritize single-process correctness before high concurrency. Once the storage engine is stable, synchronization will be introduced at the buffer-pool and transaction layers. Mutexes and read-write locks can protect shared in-memory structures, while logical database locks can control transaction-level conflicts.

A later version may investigate MVCC. MVCC is not required to demonstrate the core storage engine, and introducing it too early would obscure the relationship between basic page operations and transactional semantics. The project therefore treats concurrency as a staged subsystem rather than a prerequisite for the first persistent prototype.

15. Observability and Developer Tooling

A distinguishing feature of AtlasDB will be internal observability. Traditional database clients expose the SQL result while hiding the path taken through the storage engine. AtlasDB can expose this path for research and debugging.

- Page inspection: display page type, page identifier, free space, and decoded entries.
- B+ tree tracing: show the pages visited during a point lookup or range scan.
- WAL inspection: decode log records and transaction boundaries.
- Execution tracing: show parser, planner, executor, and storage operations.
- Buffer statistics: report hits, misses, evictions, dirty pages, and pin counts.
- Database statistics: report tree height, node occupancy, record counts, and page utilization.

Code Example 8. Example of a future AtlasDB trace command.

```
$ atlas trace "SELECT * FROM users WHERE id = 302"

Parser      -> OK
Planner     -> IndexScan(users.id)
BTree       -> root page 1
BTree       -> internal page 7
BTree       -> leaf page 42
Buffer      -> hit=3 miss=0
Executor    -> 1 row
```

16. Proposed Experimental Features

AtlasDB is intended to remain an experimental platform rather than a clone of an existing embedded database. Features will be added only when they can be isolated and benchmarked. Candidate research features include:

- Historical or time-travel queries based on record versioning.
- Native vector indexes for similarity search.
- Hybrid row and columnar storage for comparing transactional and analytical workloads.
- A visual B+ tree and page inspector.
- Adaptive buffer replacement policies.
- Alternative index implementations for controlled comparisons.
- Vectorized execution for analytical operators.

These features are deliberately excluded from the initial storage-engine milestone. Their value is experimental: they can be implemented as branches or modules and compared against the baseline engine using identical workloads.

17. Evaluation Methodology

AtlasDB will not claim performance improvements until they are measured on a defined workload and hardware configuration. The evaluation plan therefore separates correctness benchmarks from performance benchmarks.

17.1 Correctness evaluation

- Page round-trip tests: write arbitrary bytes, close the process, reopen, and verify exact contents.
- B+ tree invariant tests after random insertion and deletion sequences.
- Range-scan tests verifying sorted and complete output.
- Record serialization round trips across all supported data types.
- Transaction tests for commit and rollback behavior.
- Crash-recovery tests at selected WAL and page-write boundaries.

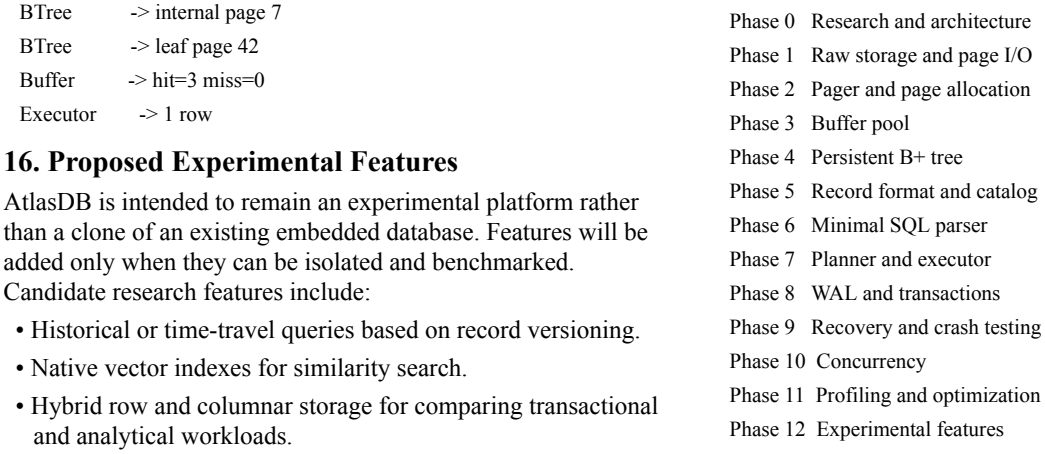
17.2 Performance evaluation

Performance experiments will measure point lookup latency, sequential range scans, insertion throughput, deletion throughput, buffer hit ratio, WAL append throughput, and recovery time. Each benchmark should report workload size, page size, cache size, hardware, Go version, build mode, and whether data is cold or warm in the operating-system cache.

Comparisons against SQLite should be treated carefully. SQLite and AtlasDB may make different durability, compatibility, query-planning, and storage-format choices. A fair comparison must therefore state configuration and workload assumptions rather than presenting a single throughput number as a general ranking.

18. Implementation Roadmap

Figure 3. AtlasDB implementation sequence.



The ordering is intentional. Each phase produces a usable lower-level abstraction that can be tested independently. For example, the B+ tree should not be implemented before page allocation and persistence are reliable, because the tree's nodes are ultimately persistent pages. Similarly, SQL should not

become the main focus before records and indexes can survive process restarts.

19. Limitations

The first AtlasDB design is not intended to compete with mature production databases. It will initially omit many features required for production use, including distributed replication, sophisticated cost-based optimization, mature security controls, extensive SQL compatibility, online schema changes, advanced isolation levels, and broad hardware portability.

The project also faces a methodological limitation: because it is being built from scratch, early design decisions may change as implementation experience reveals hidden constraints. Such changes are not treated as failures. Instead, architecture decision records should preserve the reason for each change so that the project documents the engineering process as well as the final design.

20. Discussion

The central thesis of AtlasDB is that database engineering becomes easier to understand when the abstractions are built in the same direction as the physical data path. A row stored by a SQL INSERT is ultimately represented by bytes in a page. The page belongs to a persistent structure such as a B+ tree. The page is cached by a buffer manager and written through a pager to a file. Durability adds a log and recovery protocol. Query processing then operates above these structures, selecting how to locate and transform the stored data.

This layered approach also creates a useful research environment. A change to page size can be evaluated without rewriting the parser. A new replacement policy can be compared without changing the B+ tree. A new index can be tested under the same executor. The architecture therefore supports controlled experiments in addition to normal database functionality.

The project is especially suitable for studying the trade-off between simplicity and performance. Established systems contain decades of engineering around edge cases, compatibility, and workload diversity. AtlasDB can intentionally start with a smaller problem and make each trade-off explicit. The resulting system may be less capable than a mature database but more transparent as a research artifact.

21. Conclusion

AtlasDB proposes a small but technically serious relational database engine implemented from scratch in Go. Its design starts with raw file I/O, introduces fixed-size pages and a pager, adds a buffer manager and persistent B+ tree, then builds records, catalogs, SQL parsing, query planning, execution, WAL, recovery, and concurrency on top of those foundations. The project deliberately avoids fabricated performance claims and instead defines a benchmark methodology for evaluating the implementation as it matures.

The most important outcome of AtlasDB is not a particular feature count. It is a coherent implementation in which the relationship between logical database behavior and physical storage can be inspected, tested, and explained. By making pages, indexes, logs, and execution paths observable, AtlasDB can serve as both a functioning database prototype and a long-term systems research platform.

References

[1] SQLite Project. “Database File Format.” SQLite Documentation, including page layout, B-tree pages, record format, and schema representation. https://www.sqlite.org/fileformat.html

[2] SQLite Project. “Query Planning.” SQLite Documentation. https://www.sqlite.org/queryplanner.html

[3] R. Bayer and E. McCreight, “Organization and Maintenance of Large Ordered Indexes,” Acta Informatica, vol. 1, no. 3, pp. 173–189, 1972. DOI: 10.1007/BF00288683.

[4] SQLite Project. “File Locking And Concurrency In SQLite Version 3.” SQLite Documentation. https://www.sqlite.org/lockingv3.html

[5] SQLite Project. “The Next-Generation Query Planner.” SQLite Documentation. https://www.sqlite.org/queryplanner-ng.html

[6] D. Abadi, P. Boncz, S. Harizopoulos, S. Idreos, and S. Madden, “The Design and Implementation of Modern Column-Oriented Database Systems,” Foundations and Trends in Databases, vol. 5, no. 3, pp. 197–280, 2013. DOI: 10.1561/1900000024.

[7] SQLite Project. “Write-Ahead Logging.” SQLite Documentation. https://www.sqlite.org/wal.html

[8] A. Petrov, Database Internals: A Deep Dive into How Distributed Data Systems Work. O’Reilly Media, 2019.